NeuroSonic: Implementation and Evaluation of a Laptop-Assisted, Offline Smartphone Prototype for Spoken Walking-Path Awareness and Navigation

Allwin Raj R
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Madhu Sudharshanan
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Review 3 (DA3): implementation, architecture and methodology, and result analysis of the proposed work.

***Abstract*—We implement and evaluate NeuroSonic, a research prototype in which an iPhone browser acts as the camera, microphone, GPS receiver and speaker of a perception and navigation system running on a laptop over local Wi-Fi, without internet. Four real-time detectors were benchmarked on the same 365 near-infrared images with 129 people; RF-DETR Nano gave the highest person AP@0.5 (0.660) and recall at the operating threshold (0.729) at 21.3 ms on an RTX 4050 Laptop GPU, with SSDLite as CPU fall-back. Detections are tracked with a ByteTrack-style association that compensates for camera motion estimated with Lucas–Kanade flow and RANSAC, and two conservative events, entering the walking path and crossing in front, are spoken with an offline neural voice. Offline voice commands and OpenStreetMap A* routing complete the system. In loopback browser tests the pipeline sustained about 5 frames/s with a median capture-to-answer time of 67 ms. Three serving problems were found and fixed: idle CPU threads (712 % to 121 % of one core), per-request inference threads, and GPU power states at low frame rates (mitigated with FP16 at unchanged accuracy). Ten-minute soak tests ran without errors. Informal checks on a real iPhone ran at 2.5–4.8 frames/s with 217–372 ms from camera to answer; formal field validation with users is pending; the system is not a safety device.**

***Index Terms*—assistive technology; visual impairment; electronic travel aid; obstacle detection; RF-DETR; multi-object tracking; camera-motion compensation; offline speech; pedestrian navigation**

I. Introduction

Review 1 established the need for affordable, continuous, motion-aware guidance for pedestrians with visual impairment, and Review 2 showed that the base paper's cross-modal detector [1] cannot be driven by an ordinary phone camera and that its reported accuracy cannot be reproduced with the published data. This review reports the implementation of the proposed system, **NeuroSonic**, and analyses its results.

NeuroSonic uses an iPhone as the camera, microphone, GPS receiver and speaker of a perception and navigation system that runs on a laptop over the local Wi-Fi network. Its contributions are:

1. a working end-to-end prototype: a browser client and a laptop server with 28 HTTP endpoints, providing offline walking-path guidance, event alerts, voice commands, neural speech and offline routing;
2. a controlled comparison of four real-time detectors on the same images, leading to RF-DETR Nano [2] with an SSDLite fall-back [3], [4];
3. a camera-motion-compensated tracker and two conservative, rule-based events ("entering the walking path" and "crossing in front") designed to avoid false alarms from the user's own movement;
4. a measured account of three performance problems that arise when a GPU serves a low, fixed frame rate, with before/after data; and
5. a strict separation of what was measured on the laptop, what was simulated in a browser, and what still needs an iPhone and field tests, with a protocol for the latter.

All measurements were made on one laptop (HP Omen; AMD Ryzen 7 7840HS; 15.3 GB RAM; NVIDIA GeForce RTX 4050 Laptop GPU, 6 GB) on 2–3 November 2026. The only iPhone results are informal screenshot readings (end of the results section). **No result in this review comes from a formal field test or from users with visual impairment.**

II. System Architecture

A. Deployment

The server is a standard-library Python HTTP(S) server on the laptop. The phone loads a web application (HTML, CSS and JavaScript) from it; nothing is installed on the phone. Safari grants camera and microphone access only to secure pages, so the server uses HTTPS with a certificate issued by a locally generated certificate authority. The user installs the authority once from a plain-HTTP setup page on the next port; browser security is never disabled. The deployment architecture is shown in Fig. 2 and the per-frame pipeline in Fig. 1.

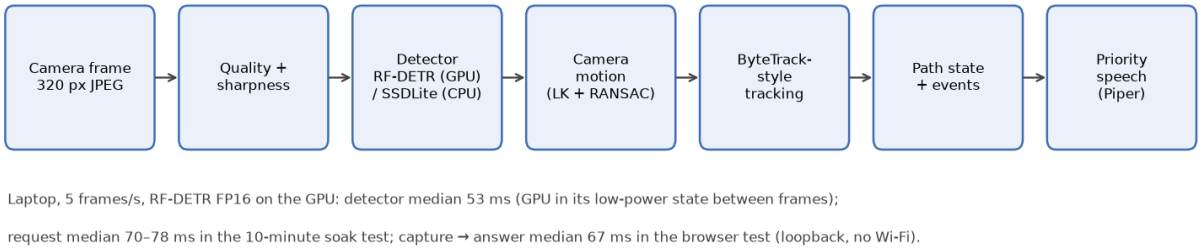


Fig. 1. Per-frame perception pipeline on the laptop: frame quality check, detector (RF-DETR on the GPU or SSDLite on the CPU), camera-motion estimation, tracking, walking-path state and events, and the phone's priority speech queue.

B. Phone client

- Posts a 320-pixel-wide JPEG every 200 ms (400 ms if the laptop reports the CPU detector), with at most one request in flight, so a slow answer reduces the frame rate instead of building a queue.
- Records speech as WAV for command recognition when the user presses the talk button.
- Uses GPS locally and sends a position only to request a route.
- Plays speech through a priority queue (urgent > system > obstacle > navigation/scene > information) with per-category cool-downs; urgent warnings interrupt routine speech, and an interrupted navigation step is replayed once.
- Shows a detector chip on the home screen (RF-DETR GPU or SSDLite CPU) so that a sighted helper can see which detector is active.

C. Laptop server

TABLE I
MAIN SERVER MODULES

Module	Responsibility
`server.py`	HTTP(S) API (16 GET, 12 POST endpoints); per-connection TLS handshakes in worker threads
`detectors.py`	RF-DETR (CUDA, FP16) and SSDLite (CPU) behind one interface with fall-back
`awareness.py`	frame pipeline, single inference worker thread, client statistics
`tracking.py`	camera-motion estimation, motion-compensated two-stage tracker
`events.py`	entering / crossing rules and their wording
`pathguide.py`	walking-path states and their wording
`tts.py`	Piper neural voice (Indian English), cached WAV with HTTP range support
`voicecmd.py`	Vosk two-pass recognition and intent parser
`maps.py`	offline OpenStreetMap graph, place search, A* routes
`store.py`	SQLite history and saved places, text only
`vision/`	optional cloud scene description (Gemini) with consent, rate limit and wording filter

D. Privacy and safety by design

Camera images are processed in memory and are never written to disk, logs or the database; the history stores text messages only. The cloud description sends one image to a third party only after an explicit opt-in. Every spoken message is drawn from a fixed vocabulary that never states a distance and never says that a way is safe; the cloud description is filtered for the same words. The application states on its "How it works" screen that it is not a replacement for a cane, guide dog or human assistance.

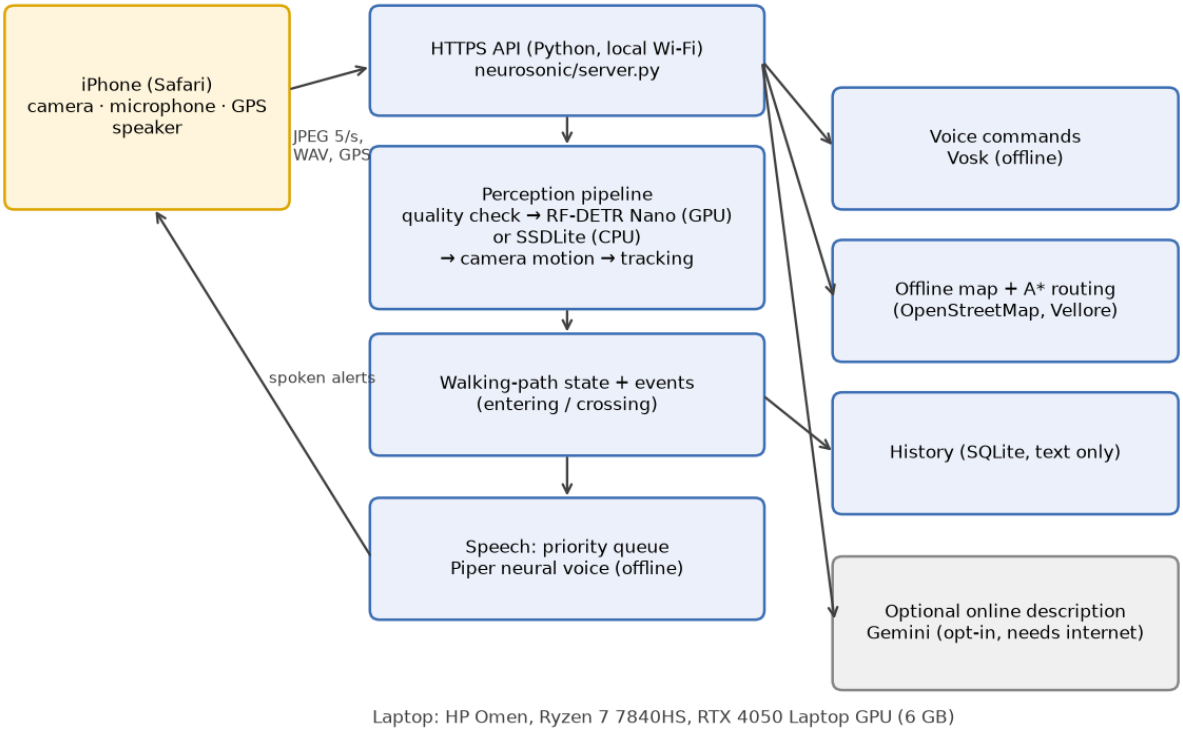


Fig. 2. NeuroSonic deployment architecture. The phone (browser) sends frames, audio and position over HTTPS on the local network; the laptop runs perception, events, neural speech, offline voice commands, offline map routing and text-only history. Only the optional scene description uses the internet, after the user opts in.

III. Methodology

A. Detector selection

Four candidates were run through the same benchmark harness: the previous production detector SSDLite320-MobileNetV3-Large [3], [4] (torchvision), RF-DETR Nano [2] (rfdetr 1.11.2), and YOLO26n and YOLO26s [5] (ultralytics 8.4.198, used for the benchmark only). All are trained on COCO [6].

- **Quality** was measured as person average precision with the same AP code used to evaluate WOAD, on 365 near-infrared validation images containing 129 annotated people. These are the only locally available images with person boxes. They are not phone RGB images, so they *rank* the detectors but do not predict street performance. Precision and recall are also reported at the production confidence threshold of 0.40.
- **Latency** was the mean over 200 (GPU) or 60 (CPU) recorded 640×480 frames, back to back, FP32, batch 1, after 10 warm-up frames.
- **Licence** was a criterion because the system is a network service: AGPL-3.0 would oblige release of the source code to its users.

B. Detector runtime and fall-back

RF-DETR runs on CUDA from local weights; downloads are blocked at run time. Detections are restricted to 23 obstacle-relevant COCO classes (e.g. person, bicycle, car, motorcycle, bus, truck, dog, cow, chair, bench, potted plant) at confidence ≥ 0.40. A wrapper falls back to SSDLite on the CPU if CUDA, the package or the weights are missing, if warm-up fails, or if a run-time inference fails, in which case the same frame is re-analysed by SSDLite. Three serving changes were made after measurement: PyTorch is limited to two CPU threads while the GPU detector is active; all inference runs on one long-lived worker thread rather than on the HTTP server's per-request threads; and RF-DETR runs in FP16 without graph compilation, with FP32 as automatic fall-back.

C. Camera motion estimation

Up to 150 Shi–Tomasi corners [7] are taken from a 320-pixel grey image, *outside* the detected boxes so that moving objects do not bias the estimate. They are followed into the next frame with pyramidal Lucas–Kanade optical flow [8], [9], and a similarity transform (translation, scale, rotation) is fitted with RANSAC [10] using OpenCV [11]. With fewer than 12 corners or inliers the camera motion is marked "unknown". The transform M maps a point in the previous frame to where a static point would appear now.

D. Motion-compensated tracking

Each track's box is predicted by applying M to its last box. Association follows the two-stage idea of ByteTrack [12], in place of SORT's Kalman prediction [13]:

- detections with confidence ≥ 0.55 are matched first, greedily by IoU, within each class;
- detections between 0.40 and 0.55 may only extend existing tracks;
- tracks end after three missed frames.

Every update stores the object's displacement relative to the background, i.e. the observed displacement minus the displacement predicted from camera motion alone:

$$\Delta_{rel} = c_t - M(c_{t-1})$$

where c is the box centre. A parked car seen while the user turns has $\Delta_{rel} \approx 0$, whereas a person stepping sideways has a large Δ_{rel}.

E. Walking-path states and events

The walking path is a lower-central trapezoid of the image. Its state is CLEAR_VIEW, POSSIBLE_OBSTACLE, BLOCKED_VIEW, UNCERTAIN (dark or blurred frame) or CAMERA_UNAVAILABLE, confirmed over 2-of-3 or 3-of-3 frames before it is spoken, with chest-mounted and hand-held wording (e.g. "The visible path appears clear. Continue carefully."). Events are considered only for confirmed tracks (at least three updates over at least 0.4 s) of people, vehicles or animals:

- *Entering:* the object was outside the path at the start of a 1.5 s window, overlaps it now, and moves towards it at ≥ 0.12 image widths per second relative to the background.
- *Crossing:* the object's centre passes from below 35 % to above 65 % of the image width (or back) within 3 s at ≥ 0.15 widths/s, with the bottom of its box in the lower half of the image.

Nothing is announced while the camera shifts more than 6 % of the width per frame, zooms more than 4 %, or has no motion estimate; nothing is announced in BLOCKED_VIEW, UNCERTAIN or CAMERA_UNAVAILABLE; each track triggers at most one event; and events are at least 3 s apart. An example message is "Caution. A person is moving into your path from the left." An "approaching" event was deliberately not implemented: with a moving camera, box growth is dominated by the user's own walking.

F. Speech, voice commands and routing

- **Speech output:** Piper [14] (VITS-based [15]) with the Indian-English voice `en_IN-spicor`; results are cached and delivered as WAV with HTTP range support (needed by iOS); the phone's own voice is the fall-back.
- **Voice commands:** Vosk [16] with the small Indian-English model, in two passes (grammar-constrained, then free), combined by an intent parser: navigate to, search, start/stop navigation, repeat, start/stop awareness, status, help, test voice. Spoken destinations are only proposed and confirmed by the user.
- **Routing:** A* [17] with a haversine heuristic on an OpenStreetMap [18] extract of Vellore (31,439 path nodes, 813 named places). Turn instructions are spoken about 70 m and 25 m ahead; the route is recalculated after two consecutive off-route positions.
- **Optional cloud description:** one photo to Gemini [19] after explicit consent, with a 30 s attempt timeout, a 45 s budget and one retry for a quick "service unavailable"; while waiting, the phone says that local guidance continues.

IV. Experimental Setup

TABLE II
Test environment

Item	Value
Laptop	HP Omen; Ryzen 7 7840HS (8 cores / 16 threads); 15.3 GB RAM; RTX 4050 Laptop GPU 6 GB; driver 592.82
Software	Windows 11; Python 3.12.10; PyTorch 2.15.0+cu132; torchvision 0.29.1; rfdetr 1.11.2; OpenCV; Piper 1.8.0; Vosk
Environments	`.venv` (CPU only, original); `.venv-gpu` (CUDA)
Detector quality data	365 near-IR validation images, 129 people (WOAD dataset)
Live-pipeline input	640×480 clip compositing a recorded person riding a bicycle over a recorded empty corridor; played as Chrome's fake camera or streamed by a soak-test tool
Browser tests	headless Chrome, iPhone-sized viewport, fake camera and microphone, DNS blocked; server on loopback

Apart from the informal iPhone readings at the end of this section, no measurement includes Wi-Fi transmission, iPhone JPEG encoding, real street scenes or users.

V. Results and Analysis

A. Detector comparison

TABLE III
Detector comparison. Latency: mean per 640×480 frame, FP32, batch 1. Quality: person, 365 near-IR images, 129 people

Detector	GPU ms	CPU ms	AP@0.5	AP@0.5:0.95	Precision @0.40	Recall @0.40	Licence
SSDLite320 (previous)	42.3	43.7	0.542	0.208	0.607	0.574	BSD-3
RF-DETR Nano (selected)	21.3	121.9	**0.660**	**0.304**	0.588	**0.729**	Apache-2.0
YOLO26n	12.4	27.8	0.619	0.268	0.633	0.628	AGPL-3.0
YOLO26s	11.8	57.5	0.628	0.292	0.600	0.651	AGPL-3.0
WOAD exp72 (reference, NIR + depth)	—	≈126–131*	0.218	0.073	—	—	see repo

* WOAD: CPU forward pass per image in batches of 8, FP32, with its own two-input preprocessing; its AP uses both modalities on the same 365 validation images (Review 2).

Table III, Fig. 3 and Fig. 4 summarise the comparison. RF-DETR Nano had the best AP and recall. Recall matters most for a warning system because a missed person is worse than an extra caution. Its 21 ms GPU latency fits a 200 ms frame interval and its licence is permissive. The YOLO26 models are faster but have lower recall and are AGPL-3.0. RF-DETR is too slow on the CPU (122 ms), so SSDLite remains the fall-back. RF-DETR's precision at 0.40 (0.588) is slightly lower than the others', so more false detections are expected; the temporal confirmation is designed to absorb them. On the same images, all four RGB detectors scored far above the WOAD checkpoint, which was trained on different scenes.

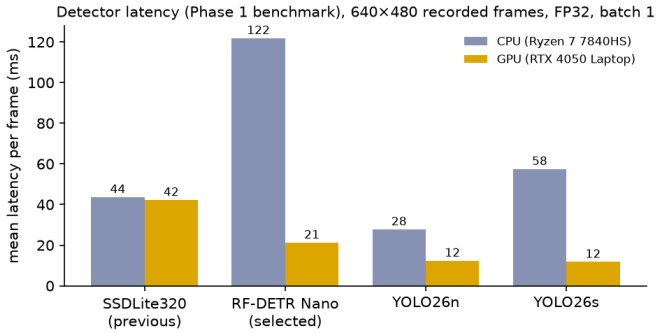


Fig. 3. Detector latency on GPU and CPU (mean ms per frame).

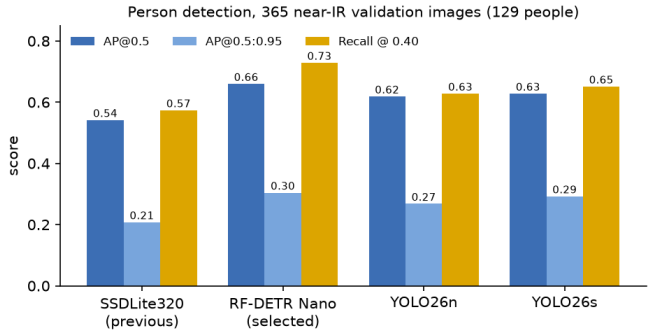


Fig. 4. Person detection quality on the near-IR validation set.

B. FP16 inference

TABLE IV
RF-DETR NANO, FP32 VS FP16 (RTX 4050 LAPTOP GPU, 320×240 FRAMES, 100 TIMED FRAMES EACH)

Precision	AP@0.5	Recall @0.40	Back-to-back med./p90 ms	At 5 fps med./p90 ms
FP32	0.660	0.729	17.3 / 21.7	66.5 / 78.6
FP16 (used)	0.660	0.729	16.9 / 20.4	53.2 / 60.8

Person quality is unchanged in FP16 (AP@0.5:0.95 0.304 vs 0.305; precision at 0.40 0.588 vs 0.595). Back to back both take about 17 ms. At the application's 5 frames/s, however, the GPU drops to its lowest power state between frames (sampled clocks 210–405 MHz), and FP16 lowers the median from 66.5 to 53.2 ms and the p90 from 78.6 to 60.8 ms (Table IV).

C. Serving efficiency at a fixed 5 frames/s

CPU threads. With RF-DETR on the GPU, PyTorch's default pool of 8 CPU threads kept the server at 712 % of one CPU core while streaming 5 frames/s, although the GPU did the detection. Two threads cut this to 121 % with no loss of frame rate (Fig. 5); median frame times were 30, 38 and 25 ms for 8, 4 and 2 threads.

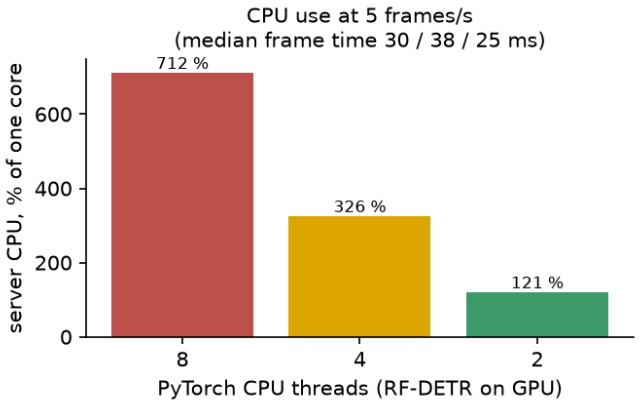


Fig. 5. Server CPU use with RF-DETR on the GPU, by PyTorch CPU thread count (712, 326, 121 % of one core).

GPU power state. At 5 frames/s the GPU is idle about 90 % of the time. In repeated timings it dropped to its lowest performance state (P8, about 390 MHz); the same inference took about 16 ms at P5 (≈600 MHz) and about 74 ms at P8. This explains why live latency is several times the back-to-back benchmark. **Inference thread.** Running GPU inference from a fresh thread per request cost about 3 ms per call in isolation (19.4 vs 16.2 ms median), so all inference was moved to one long-lived thread.

D. Ten-minute soak tests

A soak-test tool streamed the clip to the server at 5 frames/s, one request at a time as the phone does, and every 60 s sent a parallel cloud request to a mock provider that takes 20 s. "Before" ran the earlier code (FP32, inference on request threads); "after" ran the final code.

TABLE V
TEN-MINUTE SOAK TESTS, 59 WINDOWS OF 10 S

Measure	Before	After
Frames sent / errors	2980 / 0	2980 / 0
Mean frames/s	4.97	4.97
Window medians, range	62–104 ms	70–78 ms
Median of window medians	92.7 ms	74.8 ms
Highest window p90	111.3 ms	92.4 ms
Server RAM, min–max	851–2426 MiB	2255–2431 MiB
Server CPU, median (% of one core)	142	130
Parallel 20 s cloud requests answered	7 / 7	7 / 7
Events spoken	0*	148

* The first run played the clip frame by frame, so the cyclist moved at half speed (0.10 widths/s), below the 0.12 threshold; the event counts are therefore not comparable.

Both runs were stable (Table V, Fig. 6): no errors, a constant 4.97 frames/s, every slow cloud request answered while frames continued, and no growth in GPU memory. After the changes, latency was steadier (window medians 70–78 ms instead of 62–104 ms) and the worst p90 fell from 111 to 92 ms. Of the 148 events, 100 named the person and 48 the bicycle, which is correct for a person riding a bicycle. Ten minutes on one machine cannot exclude a slow leak over hours.

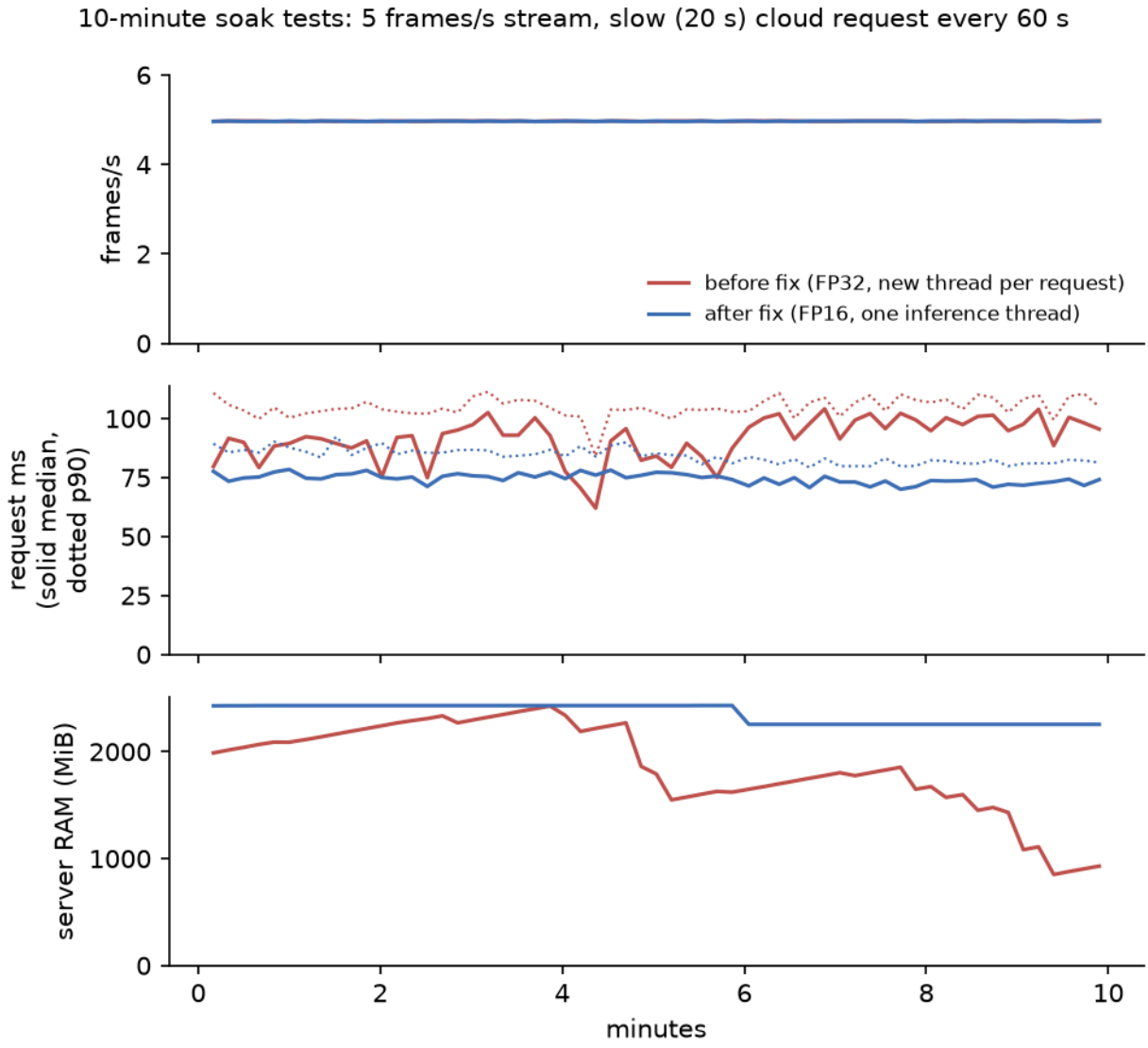


Fig. 6. Soak tests before and after the serving changes: frames per second, request latency and server memory over ten minutes.

E. End-to-end behaviour in the browser

Seven headless-Chrome tests drive the real phone interface at iPhone size against the server, with the network blocked. In the event test the page achieved 4.91 frames/s at the recommended 200 ms interval; the capture-to-answer time measured in the page (JPEG encoding, HTTP on loopback, full server pipeline) had a median of 67 ms, p90 73 ms and maximum 81 ms over 44 frames. "Caution. A person is moving into your path from the left." was delivered, and the neural-voice audio started 257 ms later. In the slow-cloud test the phone said "Still waiting for the online description. Local walking guidance continues." at 15.2 s and spoke the description at 20.1 s, while 99 frames were analysed in the meantime. Fig. 7 shows the interface.

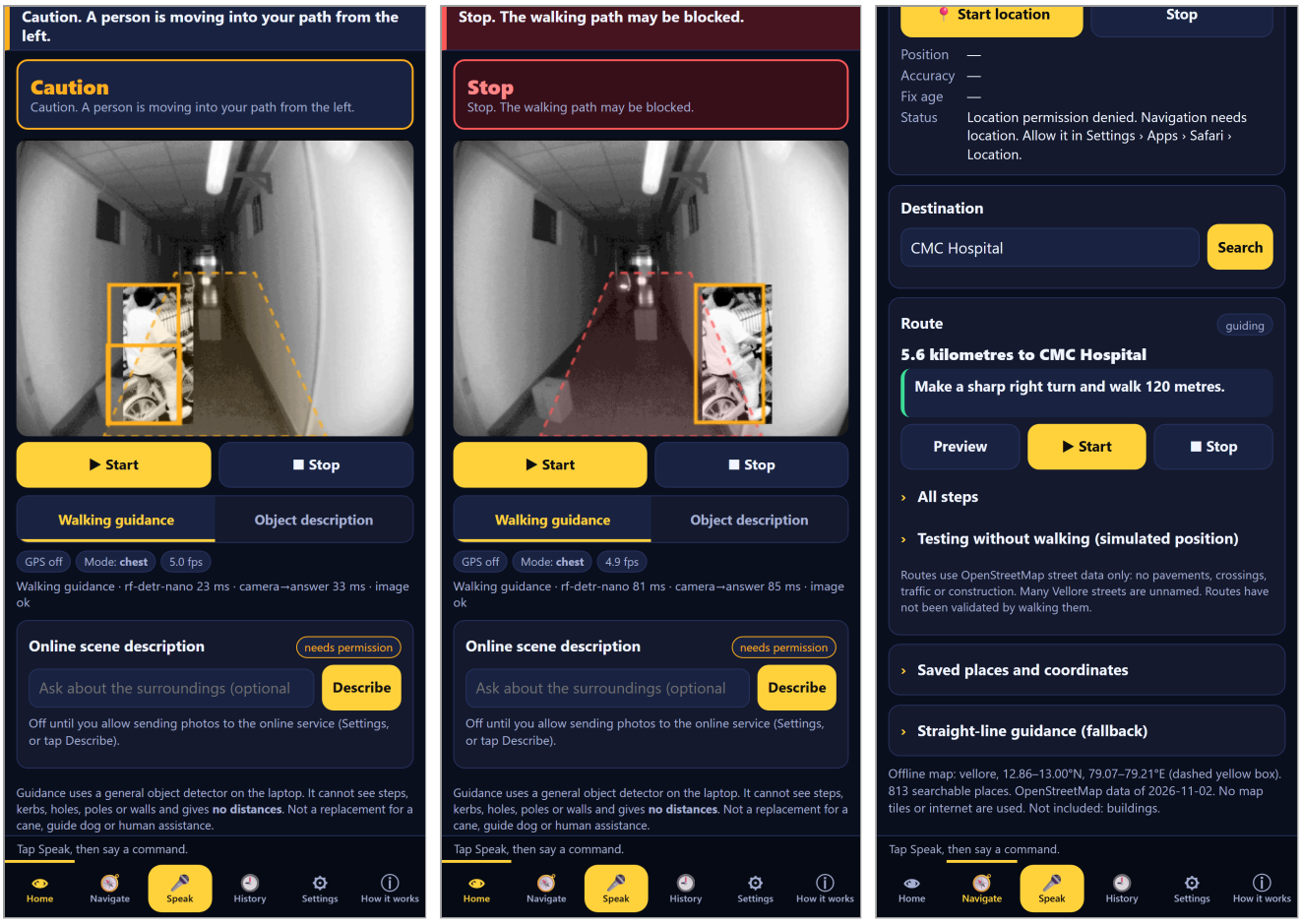


Fig. 7. Phone interface in Chrome's iPhone-sized emulation with the fake camera. Left: entering event. Centre: blocked path. Right: offline navigation with a simulated position. The camera images are a composite of recorded footage, not field footage.

F. Automated tests and quality gate

The suite runs 204 test cases (unit, integration and HTTP API tests, plus 32 Node tests of the browser logic): **OK** in the GPU environment (1 skipped: an opt-in live cloud test) and OK in the CPU-only environment (3 skipped: two GPU-only tests and the live test, with automatic fall-back to SSDLite). The 22 perception tests cover detector fall-back, recovery of a known camera shift and zoom, stable track identities while the camera pans, entering and crossing in both directions, the *absence* of events for a static car during a pan, a shaky camera, a blocked view, slow drift and chairs, and the absence of distance and safety words. All seven browser tests pass. A quality-gate launch with the documented command passed in both environments: frames, speech, place search, a 1330 m route with 10 steps, and a voice command. On 3 November 2026 the release launcher started the server as an independent background process, and its local-network HTTPS address (the one the iPhone uses) answered when checked from the laptop. An iPhone was connected afterwards for the informal check below.

VI. Informal iPhone Readings

A real iPhone ran the phone interface in Safari, connected to the laptop server over Wi-Fi (RF-DETR on the GPU). The status line under the camera image shows the frame rate, the detector time and the camera-to-answer time; Table VI lists the values read from seven screenshots. These are informal checks, not the formal 23-scenario × 5-trial validation protocol, which is still pending.

TABLE VI
Values read from seven iPhone screenshots (informal)

Scene	Banner	fps	Detector (ms)	Camera→answer (ms)
Doorway and glass partition	Caution	2.5	63	372
Furnished room	Caution	3.2	71	330
Sofa and curtain, close	Stop	3.9	62	310
Paved courtyard, parked cars	Path clear	3.8	63	217
Parked vehicle in the path	Caution	4.5	64	217
Plain wall and floor	Stop	4.8	63	233
Garage porch, motorcycle at side	Path clear	3.7	66	261
Mean		**3.77**	**64.6**	**277**

The camera-to-answer time on the iPhone (217–372 ms, mean 277 ms) is about four times the loopback median of 67 ms, because it includes JPEG encoding on the phone and the Wi-Fi transfer. All values are above the 200 ms frame interval, so the phone sends fewer frames than the 5 frames/s target; this is the main item to measure in the formal validation. The on-screen detector time is the server's per-frame figure and is not directly comparable with the 21.3 ms detector-only benchmark.

VII. Discussion

TABLE VII
Objectives and their status

Objective	Status	Evidence
O1 real time	met on the laptop	4.91–4.97 frames/s; capture→answer median 67 ms
O2 detector	met within data limits	best AP and recall of four; SSDLite fall-back
O3 motion	partly shown	synthetic-pan tests; composited clip
O4 offline	met in automated tests	network blocked in-process and in Chrome
O5 honesty	met	fixed vocabulary; wording filter; tests
O6 verification	met for the laptop; field pending	204 tests, 7 browser tests, soak tests

Serving details mattered more than the detector's benchmark speed. A detector that takes 21 ms back to back took several times longer at 5 frames/s, because of idle CPU threads, per-request threads and, above all, GPU clock scaling at low load. Benchmarks for assistive systems should therefore be run at the application's frame rate, not only back to back.

Motion without false alarms has a price. In tests with a synthetically panning camera, a static car gave no event while a person moving into the path did. But events need about one second of confirmation and are suppressed while the camera sways strongly, which can include normal walking with a hand-held phone. Missed events are therefore expected; this is acceptable only because the system supplements the cane.

Comparison with the base paper. WOAD reports field results with users and a custom device [1]; NeuroSonic reports laptop and browser results with a phone-based design that any user with a phone and a laptop could run. The two are complementary: WOAD shows what dedicated depth hardware can achieve in the field, while NeuroSonic removes the hardware barrier and adds motion events, navigation and speech input, but has yet to be validated in the field.

VIII. Limitations and Pending Validation

- **No formal iPhone or field validation yet.** RF-DETR has processed real iPhone frames only in informal screenshot checks; Wi-Fi latency, outdoor light, and use by visually impaired people are not yet measured systematically.
- **Plain surfaces.** In an informal iPhone check a textureless wall produced “Stop” although no object class was detected; the blocked-view rule treats an uninformative view as blocked. Its false-alarm rate on real surfaces is unmeasured.
- **Proxy data.** Detector quality was measured on near-infrared images with 129 people; live tests use a fake camera and a composited clip.
- **Scope.** 23 COCO classes only: no steps, kerbs, holes, poles, walls or wet floors; no distance and no "approaching" alerts.
- **Dependence.** The phone needs the laptop and local Wi-Fi; iOS Safari stops the camera when the screen locks; the GPU path needs an NVIDIA GPU.
- **Single machine.** All timings come from one laptop and driver.
- **Cloud description.** The single approved live request timed out; cloud quality and latency are unmeasured.

TABLE VIII
Physical tests still required (validation protocol: 23 scenarios × 5 trials)

#	Test	Measure
1	RF-DETR on real iPhone frames	frames processed per client, detector in use
2	Wi-Fi capture→answer latency	median / p90 ms
3	iPhone audio, microphone, voice commands	playback, recognition success
4	Real scenes with cane and sighted companion	events, misses, false alarms
5	GPS route walk in Vellore	turn timing, re-routing
6	Physically disconnected internet	all local functions
7	Live cloud description (with approval)	latency, wording
8	≥ 1 h run over Wi-Fi	errors, memory, temperature

IX. Conclusion

NeuroSonic shows that a consumer laptop GPU can give a browser-based phone client real-time, offline, spoken awareness of objects and of movement into the walking path, together with offline navigation and voice commands. In our tests it sustained about 5 frames/s and answered within roughly 50–100 ms on the laptop. The general lessons are to measure at the application's frame rate, to constrain CPU threads when the GPU does the work, and to keep GPU work on one thread. The next step is not a new model but evidence: the iPhone and field protocol, followed by monocular depth for "approaching" alerts, ground-hazard data under open licences, and a native application with on-device inference that removes the laptop dependency. NeuroSonic is a research prototype and not a safety device.

References

1. Y. Gao, D. Wu, J. Song, X. Zhang, B. Hou, H. Liu, J. Liao, and L. Zhou, "A wearable obstacle avoidance device for visually impaired individuals with cross-modal learning," *Nature Communications*, vol. 16, Art. no. 2857, Mar. 2025, doi: 10.1038/s41467-025-58085-x.
2. Roboflow, "RF-DETR: real-time detection transformer," software, Apache-2.0. [Online]. Available: https://github.com/roboflow/rf-detr
3. W. Liu *et al.*, "SSD: single shot multibox detector," in *Proc. ECCV*, 2016.
4. A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE ICCV*, 2019.
5. Ultralytics, "YOLO26 documentation," software, AGPL-3.0. [Online]. Available: https://docs.ultralytics.com/
6. T.-Y. Lin *et al.*, "Microsoft COCO: common objects in context," in *Proc. ECCV*, 2014.
7. J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE CVPR*, 1994.
8. B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. IJCAI*, 1981.
9. J.-Y. Bouguet, "Pyramidal implementation of the Lucas Kanade feature tracker," Intel Corp., Microprocessor Research Labs, Tech. Rep., 2000.
10. M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
11. OpenCV library. [Online]. Available: https://opencv.org/
12. Y. Zhang *et al.*, "ByteTrack: multi-object tracking by associating every detection box," in *Proc. ECCV*, 2022.
13. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE ICIP*, 2016.
14. Piper text-to-speech, software. [Online]. Available: https://github.com/rhasspy/piper
15. J. Kim, J. Kong, and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *Proc. ICML*, 2021.

16. D. Povey *et al.*, "The Kaldi speech recognition toolkit," in *Proc. IEEE ASRU Workshop*, 2011; Vosk offline speech recognition. [Online]. Available: https://alphacephei.com/vosk/
17. P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
18. OpenStreetMap contributors, OpenStreetMap (ODbL 1.0). [Online]. Available: https://www.openstreetmap.org
19. Google, "Gemini API documentation." [Online]. Available: https://ai.google.dev/